

API documentation

COMPLETE LEARNING GUIDE

What You'll Learn

An API without documentation only has one true expert: whoever wrote it. Documentation is the contract between the team that builds an API and every team (or external developer) that consumes it, and the industry's dominant format for expressing that contract is the OpenAPI Specification (formerly Swagger) — a machine-readable YAML or JSON description of every endpoint, its parameters, request/response schemas, and possible error responses. This guide covers how to write an OpenAPI spec, how to generate interactive documentation UIs from it, why documentation drifts out of sync with real behavior if it's maintained by hand, and how an 'API-first' workflow (writing the spec before the code) keeps the two in sync by construction rather than by discipline alone.

1. Why Documentation Is Part of the API, Not an Afterthought

From a consumer's point of view, an API's documentation is the API — nobody outside the team that wrote it can read the server source code to figure out what a field means or what status codes an endpoint can return. Undocumented or poorly documented APIs push that discovery cost onto every single consumer, repeatedly, usually via trial and error against a staging environment or by asking the original author directly. Good documentation pays for itself the moment a second team, a mobile app, or an external partner needs to integrate, and becomes mandatory the moment the original author is unavailable or has moved on.

2. The OpenAPI Specification: Describing an API as Data

OpenAPI (the specification formerly known as Swagger) describes an API as a structured YAML or JSON document: a list of paths, each with the HTTP methods it supports, the parameters each method accepts, and the schema of its request and response bodies, including error responses. Because the description is structured data rather than prose, tooling can consume it to generate interactive documentation, client SDKs in multiple languages, and even server-side request validation — all from the same single source of truth.

YAML

```
openapi: 3.0.3
info:
  title: Orders API
  version: 1.0.0
paths:
  /api/orders/{orderId}:
    get:
      summary: Retrieve a single order
      parameters:
        - name: orderId
          in: path
          required: true
          schema:
            type: string
      responses:
        "200":
          description: The requested order
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Order"
        "404":
          description: Order not found
components:
  schemas:
    Order:
      type: object
      properties:
        id:
          type: string
        status:
          type: string
          enum: [pending, shipped, delivered]
        total:
          type: number
```

3. Generating Docs From Code vs. Writing Them Separately

There are two broad approaches to producing an OpenAPI spec. 'Code-first' tools generate the spec from annotations or decorators already in the code — swagger-jsdoc reads JSDoc comments above Express routes, Spring's springdoc-openapi reads controller annotations, and ASP.NET Core's Swashbuckle reads controller attributes — keeping the spec close to the implementation with less duplicated effort. 'Spec-first' (or 'API-first') flips the order: the OpenAPI YAML is written first, as the actual design artifact reviewed by the team, and server stub code plus client SDKs are generated from it. Spec-first is more work upfront but produces a spec that's trustworthy as documentation, versus code-first specs which are only as accurate as the comments developers remembered to update.

4. Interactive Documentation UIs

A raw OpenAPI YAML file is precise but not pleasant for a human to read; Swagger UI and Redoc are the two dominant tools that render an OpenAPI document as an interactive, browsable webpage. Swagger UI additionally lets a developer execute real requests against the API directly from the documentation page ('try it out'), filling in parameters and seeing the actual response — a huge accelerant for a new integrator's first hour with an API. Both tools are typically served at a fixed path like `/api/docs`, generated live from the same spec file the server itself validates requests against.

- Swagger UI — interactive, lets developers execute real requests directly from the docs page.
- Redoc — a cleaner, three-panel read-only rendering, often preferred for public-facing API reference sites.
- Both are generated from the same OpenAPI YAML/JSON — no separate documentation to maintain.

5. Documenting Examples and Error Responses, Not Just Happy Paths

The most common documentation gap isn't missing endpoints — it's missing detail on what goes wrong. A spec that only documents the 200 response for an endpoint leaves every integrator to discover its 400/401/404/422 behavior by trial and error. Every documented endpoint should list every status code it can realistically return, with an example response body for each, especially validation error shapes, since those are what integrators spend the most time handling defensively in their own code.

6. Keeping Documentation in Sync With Reality

Hand-maintained documentation drifts from the real implementation almost immediately — a developer changes a field name under time pressure and forgets to update a separate docs page, and the two silently diverge until a confused integrator files a bug report. The fix is structural, not just discipline: generate the spec from code annotations (or generate the code from the spec) so there's only one artifact to keep accurate, and add a CI check that fails the build if example requests/responses in the spec no longer match what the running API actually returns (contract testing).

- Prefer generating the spec from code (or code from the spec) over maintaining two separate, hand-written artifacts.
- Add contract tests in CI that validate real API responses against the OpenAPI schema.
- Version documentation alongside API versions — v1 and v2 docs should be separately browsable if both are still live.

7. Postman Collections as a Complementary Tool

Postman collections are a common complement to (not usually a replacement for) OpenAPI documentation: a shareable, runnable set of example requests, often with pre-configured authentication and environment variables, that a new integrator can import and start hitting the API with immediately. Postman can also import an OpenAPI spec directly to generate a starting collection. Teams commonly maintain the OpenAPI spec as the authoritative machine-readable contract and a Postman collection as a convenient, hands-on onboarding tool built from it.

Common Pitfalls

- Writing documentation once at launch and never updating it as the API evolves, letting it silently drift from actual behavior.
- Documenting only the success (2xx) response for each endpoint and omitting the error responses integrators actually need to handle.
- Maintaining a hand-written docs page and the OpenAPI spec as two separate artifacts that inevitably fall out of sync with each other.
- Skipping request/response examples, leaving integrators to guess field formats (date formats, enum casing) through trial and error.
- Not versioning documentation alongside the API, so v1 docs disappear or get overwritten once v2 launches while v1 clients are still live.
- Treating documentation as a task for 'whoever has time' rather than part of the definition of 'done' for a new endpoint.

Key Takeaways

- OpenAPI (Swagger) is the industry-standard machine-readable format for describing a REST API's endpoints, parameters, and schemas.
- Generating the spec from code annotations (or generating code from the spec) keeps documentation and implementation from drifting apart.
- Swagger UI and Redoc render the same OpenAPI document as interactive or read-only documentation sites, with no separate content to maintain.
- Document every realistic status code an endpoint can return, with examples — not just the happy path.
- Contract testing in CI (validating real responses against the OpenAPI schema) is what keeps documentation honest over time.

Learn More

OpenAPI Specification

<https://swagger.io/specification/>

Swagger UI

<https://swagger.io/tools/swagger-ui/>

Redoc

<https://github.com/Redocly/redoc>